RAG Data Loading - Beginner Notes

RAG means Retrieval-Augmented Generation. It lets an application retrieve relevant information from its own documents and provide that information to a language model before asking the model to answer a question.

The data-loading stage is the first step in a typical RAG pipeline:

1. Choose source files
   Sources can be TXT, PDF, HTML, CSV, Word documents, web pages, or database records. In this lesson, this TXT file is the source document.

2. Load the documents
   A document loader reads a source file and converts it into a document object. A LangChain TextLoader can read a plain-text file:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("data/rag_data_loading.txt", encoding="utf-8")
documents = loader.load()
```

   The result is usually a list of Document objects. Each object contains:
   - page_content: the text that was loaded
   - metadata: information such as the source file path

3. Split the text into chunks
   Large documents are split into smaller, meaningful pieces. Chunking helps retrieval find only the relevant context and helps stay within model limits. A common approach is to use a chunk size with a small overlap so that important sentences are not cut off between chunks.

4. Create embeddings
   Each chunk is converted into a vector, called an embedding. Similar meanings produce vectors that are close to one another.

5. Store the vectors
   The vectors and their text are saved in a vector store. Examples include Chroma, FAISS, and Pinecone.

6. Retrieve and generate
   When a user asks a question, the question is embedded and compared with the stored chunks. The most relevant chunks are added to the prompt, and the language model generates an answer using that context.

Important distinction:
Loading reads the original source. Splitting prepares the loaded text for retrieval. Embedding represents chunks as vectors. Retrieval selects useful chunks at question time. These are separate stages.

Good loading practices:
- Use the correct file encoding, usually UTF-8.
- Keep the original source in metadata for citations and debugging.
- Check that the loader actually returns documents before continuing.
- Remove accidental secrets or private data before indexing files.
- Use clear headings and complete sentences when creating source documents.

Practice questions:
- What is the difference between loading and chunking?
- Why does a RAG system need metadata?
- What could happen if the source document is much larger than the model context window?
- Which parts of the pipeline happen once during indexing, and which part happens each time a user asks a question?